



Escuela Técnica Superior de
Ingeniería Informática

TRABAJO FIN DE GRADO

Smart Set Points: control de redes eléctricas con Inteligencia Artificial

Realizado por
Daniel López Ramos

Para la obtención del título de
Grado en Ingeniería Informática - Ingeniería del Software

Dirigido por
David Gutiérrez Avilés
Manuel Barragán Villarejo

En el departamento de
Lenguajes y Sistemas Informáticos

Convocatoria de junio/julio/diciembre, curso 2025/26

Aquí la dedicatoria del trabajo

Agradecimientos

Quiero agradecer a X por...

También quiero agradecer a Y por...

Resumen

Incluya aquí un resumen de los aspectos generales de su trabajo, en español.

Palabras clave: Palabra clave 1, palabra clave 2, ..., palabra clave N

Índice general

1	Introducción	1
1.1.	Contexto	1
1.2.	Motivación	1
2	Estado del Arte	2
2.1.	Redes de Distribución	2
2.2.	Técnicas de Aprendizaje Automático	2
2.2.1.	Ciclo de Vida del Aprendizaje Automático	3
2.2.2.	Modelos de Aprendizaje Automático	5
3	Objetivos	10
4	Planificación	11
4.1.	Planificación Temporal	11
4.1.1.	Tareas de Investigación	11
4.1.2.	Tareas de Análisis	11
4.1.3.	Tareas de Desarrollo	11
4.1.4.	Tareas de Evaluación	11
4.1.5.	Tareas de Cierre	11
4.2.	Planificación Financiera	11
4.2.1.	Tabla de Sueldos	11
4.2.2.	Estimación de Tiempos y Costes	11
4.2.3.	Costes Finales	11
5	Análisis	12
5.1.	Requisitos Generales	12
5.2.	Requisitos de Información	12
5.3.	Requisitos Funcionales	12
5.4.	Requisitos No Funcionales	12
6	Implementación	13
6.1.	Decisiones de Diseño	13
6.2.	Arquitectura	13
6.3.	Tecnologías y Herramientas	13
6.4.	Desarrollo	13
7	Resultados	14
7.1.	Informe de Evaluación	14
8	Conclusiones	15
8.1.	Trabajos futuros	15

9 Anexos	16
9.1. Glosario de Términos	16
9.2. Manual de Instalación	16
9.3. Manual de Usuario	16
Bibliografía	17

Índice de figuras

2.1. Principales Procesos del Aprendizaje Automático. Fuente: Elaboración propia	3
2.2. Ejemplos de subajuste, buen ajuste y sobreajuste. Fuente: [1]	4
2.3. Esquema de Random Forest y Técnica de Bagging. Fuente: Elaboración propia	5
2.4. Esquema de Árbol de Decisión. Fuente: Elaboración propia	6
2.5. Esquema de XGBoost. Fuente: [2]	7
2.6. Estructura de una Red Neuronal. Fuente: Elaboración propia	8

Índice de tablas

Índice de extractos de código

1. Introducción

1.1. Contexto

1.2. Motivación

2. Estado del Arte

En el presente apartado se ofrece una visión general del dominio del problema, abarcando tanto los fundamentos eléctricos como las técnicas de aprendizaje automático relevantes para la comprensión del objetivo del presente proyecto.

2.1. Redes de Distribución

2.2. Técnicas de Aprendizaje Automático

El Machine Learning o Aprendizaje Automático es una rama de la Inteligencia Artificial cuyo objetivo es el estudio y desarrollo de sistemas capaces de adaptar su comportamiento a partir de la experiencia. Para construir dicha experiencia, el sistema aplica un algoritmo de aprendizaje sobre un conjunto de ejemplos de entrenamiento, dotando al sistema de la capacidad de aprender y predecir resultados sin necesidad de una programación explícita.

Existen diversos tipos de algoritmos de Aprendizaje Automático, que pueden clasificarse en general en cuatro categorías principales:

- **Aprendizaje supervisado:** El sistema se entrena con un conjunto de datos etiquetados. Cada ejemplo se describe mediante un conjunto de atributos de entrada o características y una salida objetivo conocida que define la salida pretendida por el sistema. El objetivo del modelo es aprender la relación entre las entradas y las salidas para predecir correctamente la salida de nuevos ejemplos.
- **Aprendizaje no supervisado:** El sistema se entrena con un conjunto de datos sin etiquetar. De cada ejemplo se desconoce la salida esperada, por lo tanto su objetivo es identificar patrones o estructuras en los datos que ayuden a organizarlos.
- **Aprendizaje semisupervisado:** El sistema se entrena con un conjunto de datos en el que solo una parte está etiquetada. De esta forma se aprovecha tanto la información supervisada como la no supervisada para mejorar el rendimiento del modelo.
- **Aprendizaje por refuerzo:** El sistema aprende mediante la interacción con el entorno, tomando decisiones en una secuencia de estados y recibiendo recompensas o castigos según el resultado de sus acciones. El objetivo es elegir la mejor secuencia de acciones (política óptima) que maximiza la recompensa acumulada.

Este proyecto se centrará en el uso de algoritmos de **aprendizaje supervisado**. Esta elección resulta la más adecuada dado que se dispone de un conjunto de datos

etiquetados, que contiene tanto el estado de la red en cada instante (variables de entrada) como la salida objetivo correspondiente. Estos algoritmos se pueden subdividir en **clasificación**, cuando la variable objetivo referencia a un tipo discreto y en **regresión** cuando la variable objetivo referencia a un valor continuo. Considerando que los datos con los que se trabajará corresponden con valores de magnitudes físicas, típicamente voltaje e intensidad de corriente, los modelos desarrollados se centrarán en resolver un problema de regresión.

2.2.1. Ciclo de Vida del Aprendizaje Automático

Existen múltiples algoritmos y modelos de aprendizaje automático supervisado y aunque difieren en su arquitectura interna, comparten el objetivo común de ajustar una serie de **parámetros** a partir de los ejemplos de entrenamiento. En consecuencia, entrenar un modelo consiste en hallar la combinación óptima de estos parámetros que minimice la diferencia entre las predicciones realizadas y los valores reales. Este proceso de ajuste consiste fundamentalmente en minimizar una **función de pérdida** que mide la distancia entre las predicciones realizadas por el modelo y los datos de entrenamiento. La elección de una función de pérdida adecuada es crítica y depende de la naturaleza de la tarea a resolver.

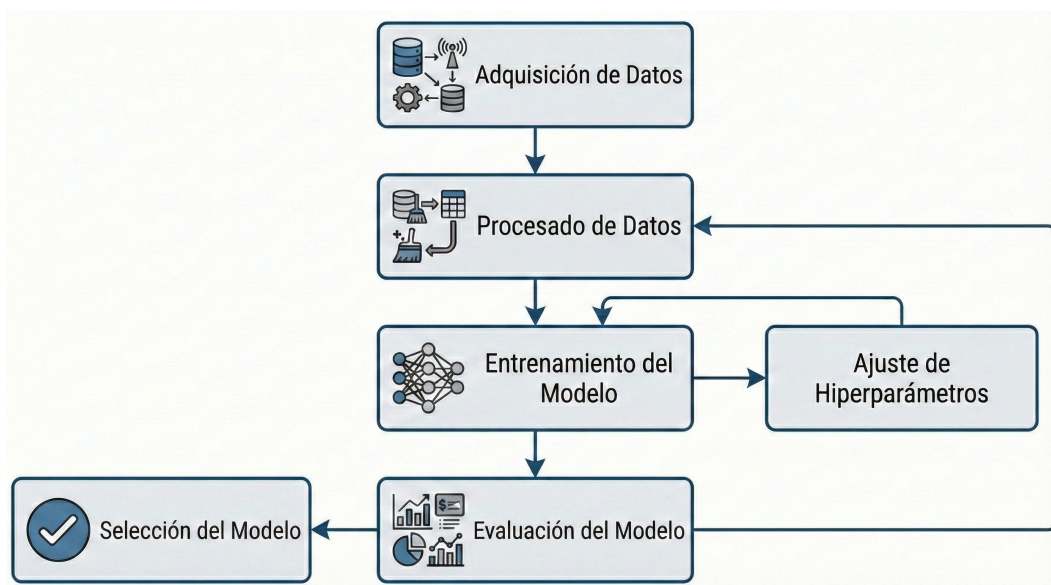


Figura 2.1: Principales Procesos del Aprendizaje Automático. Fuente: Elaboración propia

Para que el entrenamiento sea efectivo y el modelo alcance una capacidad de generalización satisfactoria, es necesario disponer de una gran cantidad de datos. La cantidad necesaria de datos varía en función de la complejidad del modelo y principalmente de la naturaleza del problema (no es lo mismo tratar de aprender un comportamiento no lineal, que uno lineal). Otro aspecto importante de los datos es la calidad de estos. Si el dataset de entrenamiento contiene errores o ruido, será más complicado para el modelo detectar patrones subyacentes. Además, los datos deberán ser representativos del dominio del problema que se pretende modelar,

teniendo gran importancia la manera en que se trabajan estos datos para no incurrir en sesgos de muestreo.

Unos de los mayores desafíos en el entrenamiento es evitar el sobreajuste o **overfitting** y el subajuste o **underfitting**. El overfitting se produce cuando el modelo se ajusta excesivamente a los datos de entrenamiento, llegando a memorizar el ruido aleatorio en lugar de aprender la tendencia subyacente. Esto resulta en una capacidad de generalización deficiente ante nuevos datos. Por el contrario, el underfitting ocurre cuando el modelo es demasiado simple o rígido para capturar la estructura y complejidad de los datos, generando predicciones imprecisas. Por tanto, el objetivo final es encontrar un punto óptimo de complejidad que permita al modelo aprender los patrones relevantes sin asimilar el ruido, garantizando así predicciones robustas y fiables ante nuevos datos.

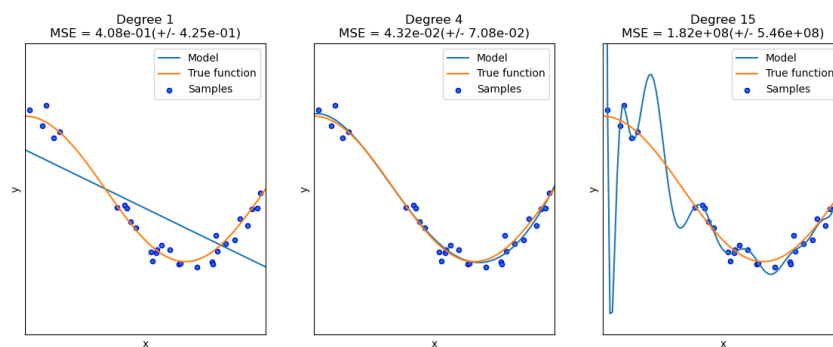


Figura 2.2: Ejemplos de subajuste, buen ajuste y sobreajuste. Fuente: [1]

Otro punto crítico en el entrenamiento de estos modelos es la **optimización de hiperparámetros**. Los hiperparámetros son variables de configuración externa que son prefijados de antemano y a diferencia de los parámetros, estos no se ajustan de manera automática durante el proceso de aprendizaje. Es fundamental en toda actividad de Machine Learning la selección de un buen conjunto de hiperparámetros, ya que estos controlan de forma directa la estructura, funciones y rendimiento de los modelos. No hay reglas establecidas en cuanto a qué hiperparámetros funcionan mejor ni sobre sus valores óptimos. Es necesario experimentar para encontrar la configuración óptima. Esta actividad se conoce como *ajuste de hiperparámetros* u *optimización de hiperparámetros* [3]. También resulta relevante para evitar el sobreajuste y subajuste, ya que establecen restricciones para los modelos, permitiendo un mayor o menor ajuste a los datos de entrenamiento.

El propósito fundamental de todo modelo de aprendizaje automático reside en la resolución eficaz de un problema específico. En consecuencia, la **validación del modelo** constituye una fase crítica para garantizar su aptitud. Este proceso requiere la definición de **métricas de evaluación** cuantitativas que midan ese rendimiento. La selección de estas métricas dependerá de las dimensiones del rendimiento que se deseen priorizar.

Para un cálculo riguroso de estas métricas, es indispensable realizar una partición del conjunto de datos original en dos subconjuntos independientes: uno destinado al entrenamiento y otro a dicho cálculo. Esta metodología proporciona

una evaluación objetiva con datos no vistos previamente por el modelo, simulando el comportamiento que tendrá el sistema en un entorno real. Dicha separación resulta esencial para detectar el sobreajuste, asegurando así que los resultados reflejen una verdadera capacidad de generalización del modelo y no la mera memorización de los patrones procesados. En siguientes secciones se explorarán las metodologías utilizadas para la selección de ese conjunto de prueba.

2.2.2. Modelos de Aprendizaje Automático

Sabiendo que el problema que se pretende resolver es de regresión, a continuación se exponen los modelos que se han aplicado en la realización de este trabajo.

Random Forest

El modelo Random Forest o Bosque Aleatorio es un algoritmo de aprendizaje supervisado que se fundamenta en el paradigma del **Ensemble Methods o Aprendizaje por Conjuntos**. Puede ser utilizado tanto en tareas de clasificación como de regresión. A diferencia de los modelos tradicionales que buscan construir un único predictor robusto, las técnicas de ensamblaje combinan las predicciones de múltiples modelos base, denominados estimadores débiles, para obtener un resultado final más preciso y estable. Esta técnica de ensamble de modelos será muy relevante para el desarrollo de este trabajo.

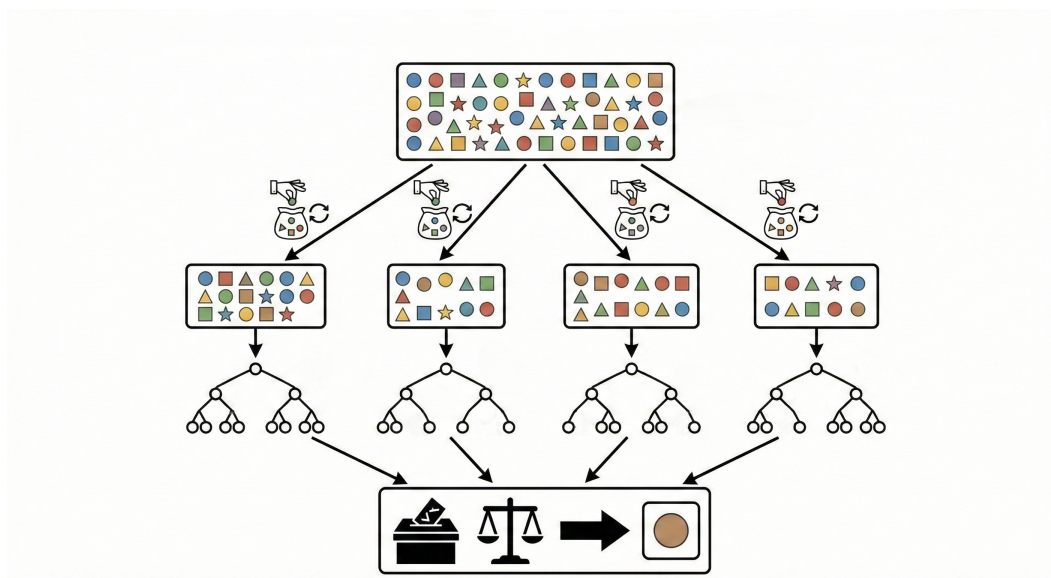


Figura 2.3: Esquema de Random Forest y Técnica de Bagging. Fuente: Elaboración propia

El enfoque de esta técnica se basa en la inteligencia colectiva. La agregación de múltiples modelos, que individualmente pueden tener un alto sesgo, tiende a compensar los errores individuales. En el caso de Random Forest, este ensamble se

construye utilizando **Decision Trees o Árboles de Decisión** como estimadores débiles, aplicando una técnica conocida como *Bagging* (Bootstrap Aggregating) [4]. Esta técnica aplicada al Random Forest, consiste en generar múltiples subconjuntos de entrenamiento del mismo tamaño que el conjunto de datos original. Este proceso se realiza mediante un muestreo aleatorio, en el que una misma instancia puede estar presente varias veces en un subconjunto y quedar excluida en otro. Posteriormente se entrena un modelo de Árbol de Decisión para cada una de estas muestras con los mismos hiperparámetros establecidos. Los modelos resultantes, por tanto, diferirán en sus parámetros, en este caso, su estructura interna. Finalmente, en problemas de regresión, la predicción del modelo global se obtiene promediando las salidas de estos modelos débiles. Esta estrategia mitiga significativamente el sobreajuste y reduce la varianza, asumiendo a cambio un mayor coste computacional proporcional al número de estimadores utilizados.

El Árbol de Decisión, algoritmo base sobre el que se fundamenta el Bosque Aleatorio, es un modelo ampliamente adoptado en el Aprendizaje Automático debido principalmente a su interpretabilidad y versatilidad. Es un algoritmo no paramétrico cuyo fundamento teórico responde a la estrategia "divide y vencerás". El modelo estructura la toma de decisiones mediante un grafo acíclico dirigido (árbol dirigido), organizando reglas lógicas secuenciales que divide el espacio de datos en regiones disjuntas. El proceso de construcción del árbol (entrenamiento) se realiza mediante un enfoque voraz y recursivo: en cada iteración, el algoritmo selecciona la partición óptima que mejor discrimina la variable objetivo. Este proceso continúa hasta satisfacer un criterio de parada predefinido, tal como alcanzar una profundidad máxima, un número mínimo de muestras por nodo o un umbral de varianza (impureza) específico.

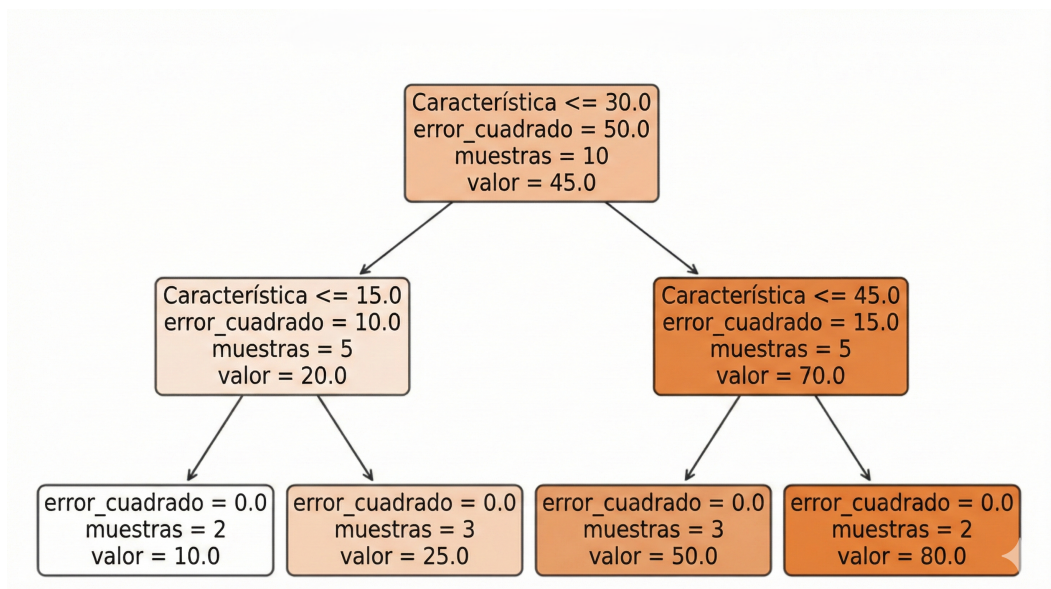


Figura 2.4: Esquema de Árbol de Decisión. Fuente: Elaboración propia

XGBoost

eXtreme Gradient Boosting (XGBoost) es una librería de Aprendizaje Automático de código abierto (open source) basada en la técnica **Gradient Boosting o Potenciación por el Gradiente** [5]. Esta metodología se basa de nuevo en los Ensemble Methods, pero a diferencia del Random Forest con un enfoque de ensamblaje paralelo, adopta una estrategia secuencial y aditiva.

En lugar de entrenar modelos independientes, el Gradient Boosting construye una cadena de estimadores débiles, típicamente Árboles de Decisión, donde cada nuevo modelo se diseña específicamente para corregir los **errores residuales** cometidos por la suma de los modelos anteriores. El término 'Gradiente' alude al mecanismo matemático de optimización subyacente: el algoritmo minimiza una **función de pérdida (loss function)** diferenciable. En cada iteración, el nuevo árbol se ajusta en la dirección del gradiente negativo de la función de pérdida respecto a la predicción actual, lo que permite refinar progresivamente la precisión del modelo global.



Figura 2.5: Esquema de XGBoost. Fuente: [2]

Redes Neuronales

Una Red Neuronal es un algoritmo basado en un proceso denominado **Deep Learning o Aprendizaje Profundo**, que utiliza nodos o neuronas artificiales interconectados en una estructura de capas que simula el comportamiento del cerebro humano. Las redes neuronales gozan de una amplia popularidad, ya que son capaces de aprender estructuras y comportamientos complejos. Son algoritmos muy usados en reconocimiento de imágenes, procesamiento del lenguaje natural o previsión de la carga eléctrica y demanda de energía entre otros [6].

La unidad mínima de procesamiento son neuronas o nodos, que están estructuradas en capas. Cada nodo recibe una entrada, la procesa y manda el resultado a nodos de la siguiente capa. Cada transferencia de una neurona previa a una posterior es modulada por un peso, que son los parámetros que el algoritmo debe aprender.

Una red neuronal básica consta de tres capas:

- **Capa de entrada:** La información exterior entra en la red neuronal artificial por esta capa, donde se procesan los datos y se pasan a la siguiente capa.
- **Capa oculta:** Toman su entrada de la capa de entrada o de otras capas ocultas. Esta capa es la que realiza la tarea de aprendizaje.
- **Capa de salida:** Proporciona el resultado final de todo el procesamiento de datos que realiza la red neuronal. Puede tener uno o varios nodos de salida.

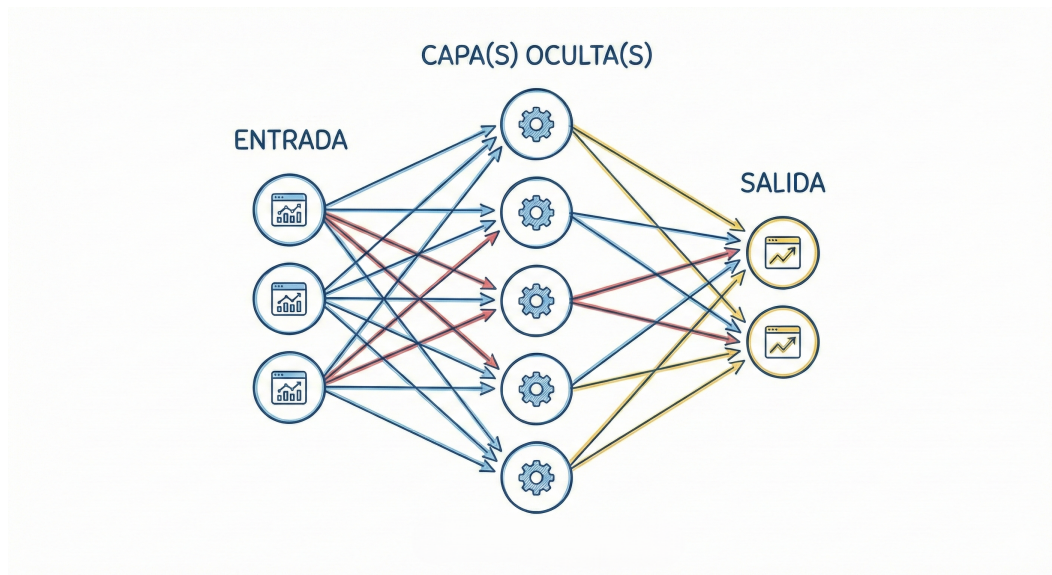


Figura 2.6: Estructura de una Red Neuronal. Fuente: Elaboración propia

Para entrenar una red neuronal, cada neurona calcula:

$$a = \sigma(W^T X + b) \quad (2.1)$$

Donde X es el vector de los argumentos, W es el vector de pesos asociados a los argumentos, b es el sesgo de activación, σ es la función de activación y a es el output [7]. La función de activación y el sesgo tienen como objetivo determinar si la neurona debe o no activarse. Esta característica es la que permite a las redes neuronales aprender patrones complejos. Las redes neuronales profundas, tienen varias capas ocultas con millones de neuronas artificiales conectadas entre sí. En teoría, estas redes pueden asignar cualquier tipo de entrada a cualquier tipo de salida. Sin embargo, también necesitan mucho más entrenamiento en comparación a otros modelos y millones de ejemplos de datos de entrenamiento.

El entrenamiento de una neurona artificial, como modelo de regresión, se realiza minimizando una función de pérdida o coste utilizando un método de

optimización como el descenso por el gradiente. Para minimizar esta pérdida, el método estándar es el algoritmo de **Backpropagation o Retropropagación**. Este algoritmo aplica recursivamente la regla de la cadena para calcular las derivadas parciales de manera eficiente de la función de coste respecto a los pesos.

Las redes neuronales han evolucionado hacia arquitecturas especializadas adecuadas para diferentes dominios. Entre los diferentes tipos de arquitectura podemos destacar:

- **Redes neuronales convolucionales:** Están diseñadas principalmente para el procesamiento de imágenes. Destacan en el reconocimiento de imágenes, la visión artificial y el reconocimiento facial gracias a filtros convolucionales.
- **Redes neuronales recurrentes:** Incorporan bucles de feedback que permiten que la información persista a lo largo de los pasos temporales. Son adecuados para el reconocimiento de voz, la previsión de series temporales y los datos secuenciales.
- **Transformadores:** Una arquitectura moderna, que aprovecha mecanismos de atención para capturar dependencias en el procesamiento del lenguaje natural y potencian modelos de última generación como GPT.

Otras Opciones

Si bien en este trabajo se ha profundizado en los modelos secuenciales y Redes Neuronales debido a su robustez y su consolidación como el estándar en la industria. Algoritmos como las **Máquinas de Vectores de Soporte (SVM)** son eficaces en espacios de alta dimensionalidad, mientras que implementaciones modernas de boosting como **LightGBM** o **CatBoost** ofrecen mejoras en eficiencia computacional y manejo de variables categóricas.

La elección final del modelo no debe basarse únicamente en la potencia bruta. No existe un algoritmo universal superior para todos los problemas. La selección depende del compromiso entre precisión, interpretabilidad, volumen de datos y recursos computacionales disponibles.

3. Objetivos

El objetivo del presente Trabajo Fin de Grado consiste en desarrollar y evaluar diferentes modelos de Inteligencia Artificial mediante aprendizaje supervisado, capaces de predecir valores de referencia (set points) que optimicen el funcionamiento de redes eléctricas de distribución. El propósito de estos modelos será predecir de manera efectiva y precisa estos valores de referencia, reduciendo la necesidad de disponer de información detallada sobre la topología y parámetros eléctricos de la red, requeridos por métodos tradicionales de optimización como el algoritmo Optimal Power Flow. Con este fin, el entrenamiento de estos modelos se realizarán a partir de grandes volúmenes de datos medidos por sensores instalados en la red.

Para alcanzar este objetivo general, se plantean los siguientes objetivos específicos:

- **OBJ-1: Análisis e Investigación de Tecnologías** - Seleccionar las librerías y frameworks de Inteligencia Artificial a utilizar y realizar un análisis e investigación exhaustiva de estas tecnologías, para una correcta implementación y optimización de los modelos predictivos en el contexto eléctrico.
- **OBJ-2: Ingeniería de Datos y Preprocesamiento** - Tratar el dataset disponible mediante tareas de limpieza, normalización y transformación, aplicando técnicas de selección de características (Feature Selection), para reducir la dimensionalidad y garantizar la calidad de los datos de entrada de los modelos.
- **OBJ-3: Construcción y Entrenamiento de Modelos Predictivos** - Desarrollar y entrenar arquitecturas de aprendizaje supervisado basadas en Random Forest, XGBoost y Redes Neuronales que se adapten a la problemática.
- **OBJ-4: Optimización de Hiperparámetros** - Aplicar estrategias de optimización de hiperparámetros sobre los modelos implementados, para maximizar métricas de rendimiento y evitar el sobreajuste (overfitting).
- **OBJ-5: Evaluación y Análisis Comparativo** - Evaluar el desempeño de los modelos utilizando métricas estadísticas rigurosas sobre conjuntos de datos de prueba no vistos anteriormente por los modelos, para realizar una comparación técnica entre estos y permitir una reflexión sobre su aplicabilidad real en contextos eléctricos.

4. Planificación

4.1. Planificación Temporal

4.1.1. Tareas de Investigación

4.1.2. Tareas de Análisis

4.1.3. Tareas de Desarrollo

4.1.4. Tareas de Evaluación

4.1.5. Tareas de Cierre

4.2. Planificación Financiera

4.2.1. Tabla de Sueldos

4.2.2. Estimación de Tiempos y Costes

4.2.3. Costes Finales

5. Análisis

5.1. Requisitos Generales

5.2. Requisitos de Información

5.3. Requisitos Funcionales

5.4. Requisitos No Funcionales

6. Implementación

6.1. Decisiones de Diseño

6.2. Arquitectura

6.3. Tecnologías y Herramientas

6.4. Desarrollo

7. Resultados

7.1. Informe de Evaluación

8. Conclusiones

8.1. Trabajos futuros

9. Anexos

9.1. Glosario de Términos

9.2. Manual de Instalación

9.3. Manual de Usuario

Bibliografía

- [1] Scikit-Learn. Explicación de underfitting y overfitting. URL https://scikit-learn.org/stable/auto_examples/model_selection/plot_underfitting_overfitting.html.
- [2] IBM. Explicación del algoritmo xgboost, . URL <https://www.ibm.com/es-es/think/topics/xgboost>.
- [3] AWS Amazon. Ajuste de hiperparámetros, . URL <https://aws.amazon.com/es/what-is/hyperparameter-tuning/>.
- [4] IBM. Bagging o bootstrap aggregating, . URL <https://www.ibm.com/es-es/think/topics/bagging>.
- [5] XGBoost. Documentación de xgboost. URL <https://xgboost.readthedocs.io/en/stable/>.
- [6] AWS Amazon. Explicación de las redes neuronales, . URL <https://aws.amazon.com/es/what-is/neural-network/>.
- [7] Aurélien Géron. Hands-on machine learning with scikit-learn, keras, and tensorflow. O'Reilly Media, 2022.